Tareas Pendientes — Kumi TTPN Admin V2

Última actualización: 2026-04-10

✅ Completado hoy (2026-04-10)

- Limpieza de archivos legacy (`comment_rails_admin.sh`, docker-compose duplicado, templates HTML ya aplicados)
- `setup.sh` ya no aborta si falla un `git clone` sin permisos
- README: versión PostgreSQL corregida a 17, documentado comportamiento de setup.sh
- **Hardcoded IDs eliminados**: `role_id == 1` → `sadmin?`, `EmployeeMovementType` constants enteras → métodos por nombre, `employee_id: 63, vehicle_id: 39` → nil
- Documentado en `ttpngas/documentacion/cambios/2026-04-10_eliminar_hardcoded_ids.md`
- `database.yml` corregido: `development` → `ttpngas_development`, `test` → `ttpngas_test`
- **Seeds completos** (`db/seeds/01` al `07`) con catálogos que reflejan valores reales de producción
- `EmployeeMovementType::NOMBRE_BAJA_LISTA_NEGRA` sincronizado con BD prod (`'Baja - Lista Negra'`)
- **Docker connectivity fix**: `LOCAL_DB_HOST=host.docker.internal` en `.env`, `host:` agregado a `database.yml`
- **54 migraciones corridas** en `ttpngas_development` (`db:migrate`)
- **Privilegios asignados**: 57 privilegios al rol `sistemas` en development
- **Documentación de migración móvil** creada:
  - `APP_MOVIL_BE_AJUSTES.md` — mapeo completo de 257 endpoints PHP → Rails REST
  - `TRAVEL_COUNTS_ANALISIS.md` — análisis campo por campo: Android → PHP → Trigger → Ruby, con 5 gaps identificados
  - `PLAN_MIGRACION_MOVIL.md` — plan en 2 fases con checklists y tiempos para el desarrollador móvil
- **Migración `20260410000002`** creada: agrega llamada a `buscar_nomina()` en el trigger `sp_tctb_insert` (Gap 1 de travel_counts). La función ya existe en Supabase — solo faltaba invocarla.

🔴 Pendiente inmediato

1. Correr la nueva migración en development y producción

```
# Development (nativo):
bundle exec rails db:migrate

# Development (Docker):
docker compose exec api bundle exec rails db:migrate

# Producción (Supabase) — confirmar mecanismo de deploy
```

Migración: `20260410000002_add_buscar_nomina_to_travel_counts_trigger.rb`
Efecto: El trigger `sp_tctb_insert` asignará `payroll_id` automáticamente en cada INSERT de travel_count.

2. Commit de toda la sesión

Nada de lo de hoy está commiteado. Incluir:

- `app/models/user.rb`, `employee_movement_type.rb`, `employee.rb`, `employee_movement.rb`, `vehicle.rb`, `concessionaire.rb`
- `app/controllers/api/v1/base_controller.rb`, `users/sessions_controller.rb`
- `config/routes.rb`, `config/database.yml`
- `db/seeds.rb` y todos los `db/seeds/0*.rb`
- `db/migrate/20260410000002_add_buscar_nomina_to_travel_counts_trigger.rb`
- `documentacion/cambios/2026-04-10_eliminar_hardcoded_ids.md`
- Documentos raíz: `APP_MOVIL_BE_AJUSTES.md`, `TRAVEL_COUNTS_ANALISIS.md`, `PLAN_MIGRACION_MOVIL.md`, `PENDIENTES.md`

3. Verificar FE tras cambios de hardcoded IDs

El análisis está en `ttpngas/documentacion/cambios/2026-04-10_eliminar_hardcoded_ids.md`. Verificar en `ttpn-frontend` que ningún componente compare directamente con `role_id == 1` o use las constantes enteras de `EmployeeMovementType`.

```
grep -rn "role_id.*1\|ALTA.*=.*1\|BAJA.*=.*2\|REINGRESO.*=.*3" ttpn-frontend/src/
```

4. Correr seeds en development

Los seeds se validaron en `ttpngas_test` limpia. Falta ejecutarlos en `ttpngas_development` para verificar que son idempotentes sobre datos reales.

```
bundle exec rails db:seed
```

🟡 Bugs conocidos (no urgentes)

5. GasCharge / GasolineCharge — callback `verificar_id` buggy

Archivo: `ttpngas/app/models/gas_charge.rb:61`, `gasoline_charge.rb:26`
Problema: `GasCharge.last` devuelve nil cuando la tabla está vacía → `nil.id` → `NoMethodError`.
Workaround actual: Seeds usan `insert_all` para saltarse el callback.
Fix real pendiente: Guardar con nil check: `actual_id = GasCharge.last; return unless actual_id`.

6. TtpnBooking — `after_create :create_actualiza_tc` ignora `$worker_status`

Archivo: `ttpngas/app/models/ttpn_booking.rb:38`
Problema: Los callbacks `before_validation` y `before_create` respetan `$worker_status = true`, pero `after_create :create_actualiza_tc` no tiene ese guard — crea TravelCounts aunque el booking sea de seed/manual.
Fix pendiente:

```
def create_actualiza_tc
  return if $worker_status
  # ... lógica actual
end
```

7. Seeds no 100% idempotentes en corridas repetidas

Tablas afectadas: `VehicleAsignation`, `TravelCount`
Causa: Usan fechas relativas (`6.months.ago`, `(i+1).days.ago`) como parte de la clave de unicidad en `find_or_create_by!` — cambian ligeramente entre corridas.
Fix pendiente: Usar fechas fijas:

```
fecha_efectiva: Date.new(2025, 10, 1)
```

8. `db:migrate` falla en DB vacía

Archivo: `ttpngas/db/migrate/20260324000003_add_missing_indexes_comprehensive.rb:70`
Problema: Intenta agregar índice sobre columna `is_active` antes de que exista.
Workaround actual: Usar `db:schema:load` en lugar de `db:migrate` para crear desde cero.
Fix pendiente: Envolver el índice en `if column_exists?`.

🔵 Gaps de travel_counts — pendientes en Ruby BE

Ver análisis completo en `TRAVEL_COUNTS_ANALISIS.md`.

Gap	Descripción	Estado
Gap 1	`buscar_nomina()` no llamada en trigger	✅ Migración creada — **falta correr**
Gap 2	Discrepancias `pnc` no se resuelven post-insert	🔴 Pendiente — agregar `after_create` en `TravelCount`
Gap 3	Override `ttpn_foreign_destiny_id` no está en backend	🟡 Pendiente — campo en `client_branch_offices` o lógica en controller
Gap 4	`created_by_id` siempre queda en 1 (trigger default)	🟡 Pendiente — pasar `current_user.id` desde controller

Gap	Descripción	Estado
Gap 5	App genera ID en cliente (fetch max+1)	🔵 Se elimina en Fase 2 migración móvil

---

🔵 Módulos y features en progreso

9. Módulo de Empleados (FE)

Ver `ttpngas/documentacion/TODO.md` — pendiente desde antes de esta sesión.

- EmployeesPage.vue con tabs
- Nested forms (Documentos, Salarios, Movimientos)

10. Catálogos de Empleados en Settings (FE)

- Employee Document Types
- Employee Movement Types
- Labors (Puestos)
- Drivers Levels

11. GasStation seeds — claves dev vs prod

Los seeds crean estaciones con clv `GAS-001..005` (inventadas). Las reales tienen clv tipo `P7_CIMARRON`, `CHI-NG1`, etc.
Opción: Cambiar a las claves reales o usar prefijo `SEED-GAS-001`.

12. Controladores Rails faltantes (requeridos por app móvil — Fase 2)

Ver `APP_MOVIL_BE_AJUSTES.md` para lista completa. Pendientes:

- `CooTravelRequestsController`
- `CooTravelEmployeeRequestsController`
- `CrdHrsController`, `CrdhRoutesController`, `CrdhrPointsController`, `CrDaysController`
- `BroadcastReceiversController`, `ClientEmployeesController`, `FixedRoutesController`
- Acciones: `firebase_token`, vehicle_check por tipo, travel_count `authorize`/`reject`

📋 Referencia rápida — comandos clave

```
# Crear DB test desde cero y semillar
RAILS_ENV=test bundle exec rails db:schema:load db:seed

# Correr migraciones (Docker)
docker compose exec api bundle exec rails db:migrate

# Después de migrar a un ambiente existente (ej: development con datos
reales)
docker compose exec api bundle exec rails runner "load
Rails.root.join('db/seeds/privileges.rb')"
```

```
# Luego asignar al rol sistemas (case-insensitive):
docker compose exec api bundle exec rails runner "
  rol = Role.find_by('lower(nombre) = ?', 'sistemas')
  Privilege.active.each { |p| RolePrivilege.find_or_create_by!(role_id:
rol.id, privilege_id: p.id) { |rp| rp.assign_attributes(can_access: true,
can_create: true, can_edit: true, can_delete: true, can_clone: true,
can_import: true, can_export: true) } }
  puts 'Listo: ' + RolePrivilege.where(role_id: rol.id).count.to_s + '
privilegios'
"

# Semillar development (idempotente)
bundle exec rails db:seed

# Terminar conexiones activas a test antes de drop
psql -c "SELECT pg_terminate_backend(pid) FROM pg_stat_activity WHERE
datname = 'ttpngas_test' AND pid <> pg_backend_pid();"
RAILS_ENV=test bundle exec rails db:drop db:schema:load db:seed

# Credenciales de prueba
# admin@ttpn.com.mx      / Ttpn8869*
# capturista@ttpn.com.mx / Ttpn8869*
# rh@ttpn.com.mx         / Ttpn8869*
```